

Ecommify:

Arquitectura híbrida PostgreSQL + MongoDB

Diseñamos una arquitectura de persistencia políglota para soportar simultáneamente cargas transaccionales y analíticas en una plataforma de *e-commerce* basada en el dataset Olist.



PostgreSQL



MongoDB



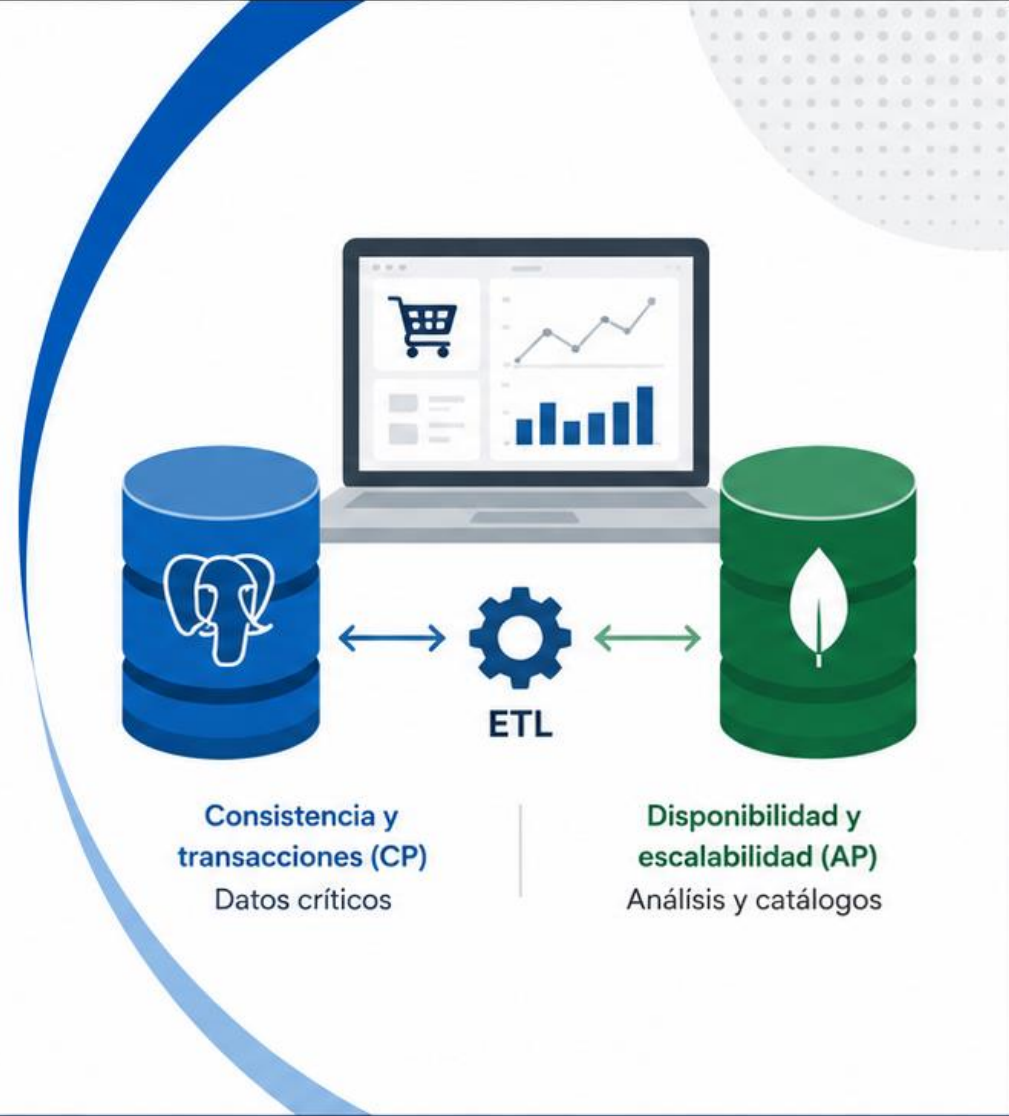
HTAP



CAP



Benchmarking



Universidad de
La Sabana

MAESTRÍA EN ARQUITECTURA DE SOFTWARE
CON ÉNFASIS EN OPTIMIZACIÓN DE BASES DE DATOS



Integrantes

- Myriam Andrea Martínez Fontecha
- Juan Francisco Javier Pérez Rivero
- Fredy Orlando Pulido Quintero
- Nicolás Felipe Torres Amaya



Proyecto Final

Grupo C1-2025-MAS
Junio de 2026



Objetivo

Demostrar, mediante evidencia experimental, que una arquitectura híbrida optimiza el rendimiento y la escalabilidad de extremo a extremo.

Arquitectura Híbrida

PostgreSQL (CP) + MongoDB (AP)

Combinamos la fortaleza transaccional y consistente de **PostgreSQL** con la flexibilidad y escalabilidad analítica de **MongoDB**.



PostgreSQL (CP)

Sistema transaccional

- Consistencia fuerte (ACID)
- Integridad referencial
- Relaciones complejas
- Datos críticos del negocio



MongoDB (AP)

Sistema analítico y flexible

- Alta disponibilidad
- Escalabilidad horizontal
- Esquema flexible
- Consultas analíticas y catálogos

FUENTE DE VERDAD TRANSACCIONAL



orders

order_items

order_payments

customers

sellers

products

product_category_name_translation

order_reviews

inventory

Tablas relacionales con integridad y transacciones ACID



ETL

- Extracción incremental
- Transformación y agregación
- Carga hacia colecciones analíticas



ETL ejecutado en notebooks de Google Colab

PROYECCIONES ANALÍTICAS Y FLEXIBLES



reviews

Reseñas de productos



geolocation

Datos geoespaciales de vendedores y envíos



products_catalog

Catálogo flexible de productos



orders_summary

Proyección analítica pre-agregada (ETL)

Colecciones documentales para consultas analíticas y catálogos



Objetivo: Maximizar consistencia en datos críticos, disponibilidad y rendimiento en analítica.



Ecommify

Plataforma de e-commerce

Optimización Implementada

Aprovechamos las capacidades avanzadas de cada motor para maximizar rendimiento y escalabilidad.



PostgreSQL (CP)

Motor transaccional y consistente



Tipos Avanzados

JSONB, TSTZRANGE, ENUM, Composite Types para modelado flexible y eficiente.



PostGIS

Consultas y cálculos geoespaciales para análisis de origen y destino de envíos.



pg_trgm

Búsqueda tolerante a errores y autocompletado de productos y categorías.



Particionamiento RANGE (orders)

Particionamiento anual por order_purchase_timestamp para mejorar rendimiento y mantenimiento.



Vistas Materializadas

Dashboards pre-computados para métricas de ventas y desempeño.



Índices Especializados

B-Tree, GIN, GiST, BRIN según patrón de consulta y volumen de datos.

BENEFICIOS OBTENIDOS



Mayor rendimiento en consultas

Reducción significativa de tiempos de respuesta.



Consistencia y confiabilidad

Propiedades ACID para datos críticos.



Escalabilidad preparada

Particionamiento y estrategias de escalado.



Optimización de costos

Aprovechamiento eficiente de recursos cloud (free tier).



MongoDB (AP)

Motor analítico y flexible



Aggregation Pipelines

Consultas analíticas complejas con \$match, \$group, \$lookup, \$facet y \$project.



Índices Compuestos (ESR)

Equality, Sort, Range para acelerar consultas de filtros y ordenamientos.



Índices Geoespaciales (2dsphere)

Consultas de proximidad y análisis geográfico de vendedores y clientes.



Replica Sets

Alta disponibilidad y tolerancia a fallos en entorno distribuido.



Estrategia de Sharding (diseño)

Preparado para escalar horizontalmente por key de partición.



ETL Analítico

Proyección pre-agregada (orders_summary) para dashboards y reportes.



Enfoque HTAP: Separación inteligente de cargas transaccionales (OLTP) y analíticas (OLAP) para evitar contención y maximizar el rendimiento global.



Ecommify

Plataforma de e-commerce

Pruebas de Rendimiento: Concurrencia

Comparación de Throughput y Latencia (p95) por Nivel de Concurrencia

METODOLOGÍA



Carga concurrente
1, 5, 10, 15, 20 y 30 usuarios



Métricas capturadas
Throughput (ops/s), latencia p50, p95, p99 y errores



Mismo framework
de pruebas para ambos motores



Ambiente
Supabase Free (PostgreSQL 16)
MongoDB Atlas M0

CONSULTAS EVALUADAS

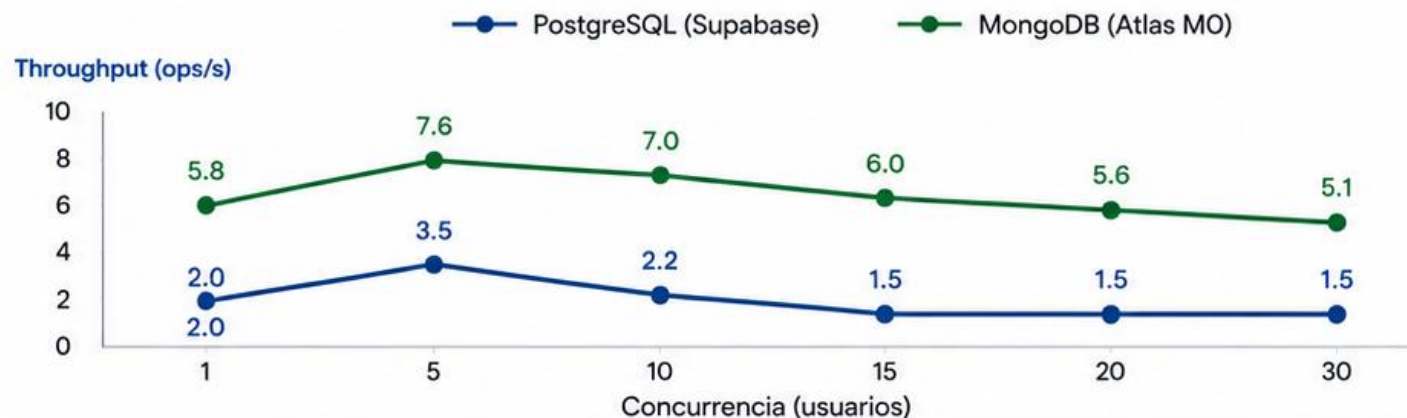


PUNTO (OLTP)
Búsqueda por ID de orden con JOINs necesarios



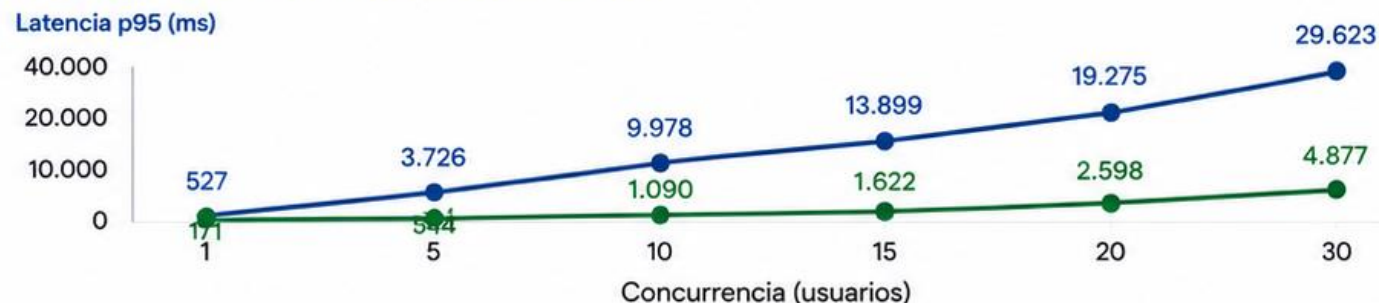
COMPLEJA (ANALÍTICA)
Métricas de ventas por mes, categoría y estado

THROUGHPUT (OPS/S) POR NIVEL DE CONCURRENCIA



MongoDB mantiene **mayor throughput** en todos los niveles de concurrencia.

LATENCIA P95 (MS) POR NIVEL DE CONCURRENCIA



PostgreSQL muestra aumento abrupto de latencia a partir de 10-15 usuarios.



Resultado general: MongoDB presenta mejor comportamiento bajo carga concurrente, con mayor throughput y menor latencia en todos los niveles evaluados.



Objetivo:

Comparar el rendimiento de ambos motores bajo diferentes niveles de carga concurrente.



Consultas:

- Punto (OLTP)
- Compleja (Analítica)



Conclusión clave:

MongoDB mantiene estabilidad y baja latencia; PostgreSQL degrada significativamente a partir de 10-15 usuarios.

Hallazgo Principal: El problema **NO** era PostgreSQL

El cuello de botella real: Saturación del connection pool de Supabase Free Tier

¿QUÉ OBSERVAMOS INICIALMENTE?



PostgreSQL mostró una degradación abrupta a partir de 5-10 usuarios concurrentes:

20x menos

throughput vs
MongoDB

24x más

latencia p95 vs
MongoDB



¿QUÉ DESCUBRIMOS?

El análisis exhaustivo de los datos reveló que el cuello de botella **NO** era el motor PostgreSQL.



El problema era el **connection pooler** del Free Tier de Supabase.

EVIDENCIA CLAVE: PATRÓN FIFO DEL POOLER

El tiempo de espera crece linealmente con la concurrencia, comportamiento típico de una cola FIFO.

Usuarios Concurrentes	Conexiones Disponibles (Pooler Free)	Espera en Cola (s) Observada	Tiempo Espera por Usuario (s)	Patrón Identificado
1	15	0 - 0.1 s	~0 s	✓ Sin espera
5	15	0.5 - 1 s	~0.1 - 0.2 s	✓ Sin espera
10	15	2 - 3 s	~0.2 - 0.3 s	✓ Cola corta
15	15	≈ 14 s	≈ 0.9 - 1.0 s	⌚ Inicio de cola
20	15	≈ 19 s	≈ 0.95 s	⌚ Cola creciente
30	15	≈ 29 s	≈ 0.97 s	⌚ Cola larga



La espera total $\approx (\text{Usuarios} - \text{Conexiones disponibles}) \times \text{tiempo por conexión}$
Este comportamiento **NO** es propio del motor PostgreSQL, sino del pooler.

¿POR QUÉ NO ES UN PROBLEMA DE POSTGRESQL?



Las consultas están optimizadas

Índices, particionamiento, vistas materializadas y tipos avanzados demuestran buen desempeño en baja concurrencia.



El colapso ocurre antes de saturar el motor

PostgreSQL puede manejar mucho más de 10 conexiones concurrentes con la infraestructura adecuada.



El patrón FIFO es inequívoco

El crecimiento lineal del tiempo de espera coincide exactamente con una cola de conexiones limitada.



Limitación del Free Tier (Supabase)

El pooler gratuito tiene un límite de conexiones concurrentes que se convierte en el cuello de botella.

CONCLUSIÓN DEL HALLAZGO



El rendimiento observado **NO** refleja las capacidades reales de PostgreSQL, sino la **restricción de infraestructura** del entorno utilizado.



RECOMENDACIÓN INMEDIATA: Migrar a un plan con mayor capacidad de pooler (Supabase Pro o superior) o implementar PgBouncer dedicado para liberar el verdadero potencial de PostgreSQL en producción.



Ecommify
Plataforma de e-commerce



Decisión basada en evidencia
Los datos nos permitieron identificar la causa raíz real del problema.



Infraestructura importa
El rendimiento puede estar limitado por la plataforma, no por la tecnología.



Arquitectura validada
La estrategia híbrida PostgreSQL + MongoDB fue la correcta.



Mejora continua
Este hallazgo nos guía hacia decisiones de escalamiento más efectivas.

1



La persistencia políglota fue la decisión correcta.

Cada motor se utilizó donde aporta mayor valor según el tipo de carga y los requisitos del negocio.

2



PostgreSQL es la mejor opción para datos críticos (CP).

Garantiza integridad, consistencia ACID, relaciones complejas y operaciones transaccionales confiables.

3



MongoDB es la mejor opción para cargas analíticas y flexibles (AP).

Ofrece alta disponibilidad, escalabilidad horizontal y un modelo flexible ideal para analítica y catálogos.

4



El principal cuello de botella encontrado fue de infraestructura, no de tecnología.

La saturación del [connection pool](#) del free tier de Supabase limitó el rendimiento observado de PostgreSQL.

5



Las decisiones arquitectónicas fueron validadas con evidencia experimental propia.

Pruebas rigurosas, comparables y reproducibles que sustentan nuestras conclusiones.

POSTGRESQL (CP) vs MONGODB (AP) — RESUMEN COMPARATIVO



PostgreSQL (CP)

Órdenes, pagos, clientes, inventario, catálogo



Casos de uso

Consistencia fuerte (ACID)



Modelo de Consistencia (CAP)

Transacciones, integridad referencial, relaciones



Fortalezas

Carga analítica masiva, escalamiento horizontal



Limitaciones

OLTP / Transaccional



Fortaleza principal



MongoDB (AP)

Reseñas, geolocalización, dashboards, catálogos flexibles

Alta disponibilidad y tolerancia a particiones

Flexibilidad de esquema, escalabilidad horizontal, consultas analíticas

Transacciones multi-documento complejas, JOINS nativos

OLAP / Analítico y flexible

IMPACTO DEL PROYECTO



Aprendizaje profundo

Comprendimos en la práctica las diferencias reales entre motores relacionales y NoSQL en un contexto HTAP.



Decisiones basadas en datos

El análisis empírico nos permitió distinguir problemas de infraestructura vs. problemas de arquitectura.



Arquitectura robusta y escalable

La combinación CP + AP permite atender las necesidades actuales y futuras del negocio.



Trabajo colaborativo efectivo

Metodología, pruebas y análisis con un enfoque riguroso, reproducible y profesional.



Este proyecto demuestra que el éxito de una plataforma no depende solo de elegir la tecnología correcta, sino de entender el problema, medir con rigor y tomar decisiones arquitectónicas informadas.



Resultado final: Una arquitectura híbrida que combina lo mejor de ambos mundos para ofrecer **consistencia** donde importa y **escalabilidad** donde se necesita.



Ecommify

Plataforma de e-commerce



Con esta presentación cerramos nuestro proyecto
Ecommify de arquitectura híbrida **PostgreSQL + MongoDB**.



Agradecimientos



A nuestro profesor por su guía, retroalimentación y apoyo durante el desarrollo de este proyecto, que nos permitió crecer como arquitectos de software.



Lo que nos llevamos



Decisiones informadas

Elegir la tecnología correcta para cada necesidad.



Evidencia experimental

Medir, analizar y validar antes de concluir.



Comprensión profunda

Infraestructura, arquitectura y datos están conectados.



Resultados significativos

Una arquitectura híbrida que entrega valor real.



Trabajo en equipo

Colaboración que hizo posible este proyecto.

“Gracias por acompañarnos en este viaje de aprendizaje, análisis y arquitectura.”